AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection

Krisha Chawan
Computer Engineering
Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS
Mumbai, India
krishamchawan@gmail.com

Manav Gangar
Computer Engineering
Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS
Mumbai, India
manavgangar2@gmail.com

Jude Dsouza
Computer Engineering
Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS
Mumbai, India
judeds2005@gmail.com

Neel Vora
Computer Engineering
Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS
Mumbai, India
neelv2005@gmail.com

Sofia Francis
Computer Engineering
Mukesh Patel School of Technology Management and Engineering, SVKM's NMIMS
Mumbai, India
sofia.francis@nmims.edu

***Abstract*—Vulnerabilities in software constitute great security threats, whereas the findings provided by conventional Static Application Security Testing (SAST) tools require substantial human effort to analyze. This paper presents an AI-assisted Security Copilot that combines SAST, machine-learning-based vulnerability classification, Retrieval-Augmented Generation (RAG), and automated patch verification. Semgrep and Bandit perform rule-based static analysis, while a fine-tuned CodeBERT model classifies Python source-code functions as vulnerable or non-vulnerable. The PyCode-Vul dataset is cleaned and evaluated for duplicates, data leakage, and overlapping samples before training. A sliding-window tokenization strategy is employed to handle functions exceeding CodeBERT's 512-token limit. To support security analysis and remediation, the RAG module retrieves contextual information from MITRE's Common Weakness Enumeration (CWE) version 4.20 using BAAI general-purpose embeddings and FAISS-based similarity search. The retrieved security knowledge supports vulnerability explanations and remediation recommendations. On the evaluated test set, CodeBERT achieves 96.88% accuracy, 95.76% precision, 97.38% recall, 96.56% F1-score, and 0.9933 ROC-AUC.**

***Index Terms*—Software Vulnerability Detection, CodeBERT, Source Code Analysis, Retrieval-Augmented Generation (RAG), Automated Vulnerability Remediation, Static Application Security Testing (SAST)**

I. Introduction

Software vulnerabilities pose major challenges to the security, reliability, and maintainability of current software systems. With the increase in software complexity, the early detection of vulnerabilities during the software development lifecycle is crucial in terms of minimizing security issues and costs. Traditional approaches such as manual code review, static analysis, and rule-based methods can highlight known vulnerability patterns, but they can produce false positives and struggle with complex or previously unseen vulnerabilities. Recent advances in artificial intelligence have enabled automated source-code analysis, with pretrained transformer models such as CodeBERT providing contextual representations for vulnerability classification.

However, vulnerability detection alone does not provide a complete security solution. The findings from static-analysis need to be validated, and the automated process needs to create security-aware patches and ensure that vulnerabilities have been patched without impacting functionality. These challenges motivate an AI-assisted pipeline that integrates static analysis, CodeBERT-based vulnerability classification, retrieval-augmented security knowledge, LLM-assisted remediation, and post-repair verification. In this work, CodeBERT is fine-tuned for Python vulnerability classification using the PyCode-Vul dataset, which is systematically preprocessed to improve the reliability of model evaluation. The resulting classifier is integrated into the broader vulnerability analysis workflow to support detection,contextual analysis, remediation, and verification.

II. Literature Survey

A. Existing Approaches to Vulnerability Detection, Classification, and Remediation

Rule-based SAST tools such as Bandit, Semgrep, and CodeQL remain the first line of defense in most pipelines, but false positives are a persistent complaint. A 2026 industrial study at Tencent [3] put the figure at 76% of enterprise alarms, tracing the problem back to LLMs still struggling with long code contexts even when used for triage. Several recent efforts try to fix this at the re-ranking stage rather than the detection stage itself. Kharkar et al. [1] improved precision by 17.5 points on null-dereference and resource-leak bugs using a

transformer-based re-ranker, and Xiong and Zhang [2] pushed the false-positive rate on the OWASP Benchmark down from over 92% to as low as 6.3% with LLM-based coding agents. ZeroFalse [4] takes a similar view, using LLMs as adjudicators over static-analyzer output rather than as detectors in their own right, and reports F1 scores above 0.9 on two benchmarks. The pattern across all four is the one this project borrows: re-rank or adjudicate existing SAST output instead of replacing it, which is the basis of its SAST-plus-classifier fusion strategy.

A second strand treats vulnerability detection as a direct code-classification problem instead. CodeBERT and its data-flow-aware variant GraphCodeBERT [6] are the dominant backbone here, and most later work builds on top of them rather than starting fresh. DFEPT [5] adds explicit data-flow embeddings to these backbones and improves accuracy on Devign and F1 on ReVeal; VulD-CodeBERT [7] wraps CodeBERT in a BiLSTM and attention ensemble and reaches 90.30% accuracy on C/C++ code. StagedVulBERT [8] goes a step further, handling both function-level detection (92.26% F1) and statement-level localization in one model. Taken together, these results are why fine-tuned CodeBERT and GraphCodeBERT variants were chosen as the backbone for this project's own Vulnerability Classifier. Much vulnerability detection research relies on Big-Vul, a C/C++ dataset whose label noise and duplication have raised concerns about evaluation reliability. Chen et al. [10] introduced DiverseVul to address these issues, while Ding et al. [11] demonstrated a substantial performance drop from 68.26% F1 on Big-Vul to 3.09% on PrimeVul. Safdar et al. [9] combined eight datasets to study cross-dataset generalization. However, these benchmarks are not Python-specific. VUDENC [12] provides a Python-focused alternative, motivating the use of PyCode-Vul [13], on which fine-tuned CodeBERT achieved 93.99% accuracy and 93.84% F1-score after ten epochs.

The third strand starts after the detection phase, addressing explanation, repair generation, and verification. Vul-RAG [14] is a knowledge-level RAG framework, which increases the pairwise accuracy by 16-24 percentage points over the baseline LLMs, and as a side effect, discovers ten new Linux kernel vulnerabilities that were not known before. As for the repair, VulRepair [15] trains a T5 model to achieve 44% Perfect Prediction rate, however, Al-Maamari's audit [16] of 319 patches generated by the LLM across 64 Java CVEs tells a more cautionary story: only 24.8% of patches were perfect and almost all failures were caused by the wrong interpretation of the code semantics, rather than the syntax errors. This is essentially the problem addressed by Cheng's Detect-Repair-Verify pipeline [17], demonstrating the positive effect of iterative re-verification on the yield of secure and correct patches. The point is that the unverified patch generated by the LLM cannot be trusted, which is the reason why this project uses the re-scan verification loop.

B. Research Gap

Three things stand out once these strands are put side by side. First, static analysis and machine-learning detection look complementary rather than competing: SAST tools give deterministic, explainable rule coverage, and transformer-based classifiers such as CodeBERT and GraphCodeBERT pick up patterns that fixed rules simply cannot express [1]–[8]. Second, reported detection performance turns out to be quite sensitive to benchmark quality: label noise, duplication, and leakage inflate results on widely used C/C++ corpora such as Big-Vul, which is exactly why cleaner, Python-native alternatives such as VUDENC and PyCode-Vul were chosen for this project [9]–[13]. Third, retrieval-augmented explanation and LLM-based repair clearly help developers, but on their own, automatically generated patches keep turning out unreliable unless something independently re-verifies them [14]–[17]. What none of the reviewed work does, though, is bring static analysis, machine-learning classification, retrieval-augmented explanation, and automated patch verification into one evaluated pipeline for Python source code. That is the gap this project tries to close with its AI Security Copilot: a Semgrep-and-Bandit SAST layer, a fine-tuned CodeBERT Vulnerability Classifier, CWE-grounded RAG explanation, and a mandatory re-scan verification loop, all inside a single unified workflow.

III. Proposed Work

A. System Architecture

As illustrated in Fig. 1, the proposed system is designed as an AI-assisted security copilot that combines static code analysis, vulnerability classification based on machine learning, retrieval-augmented generation, automated patch testing, and result visualization into a single pipeline.

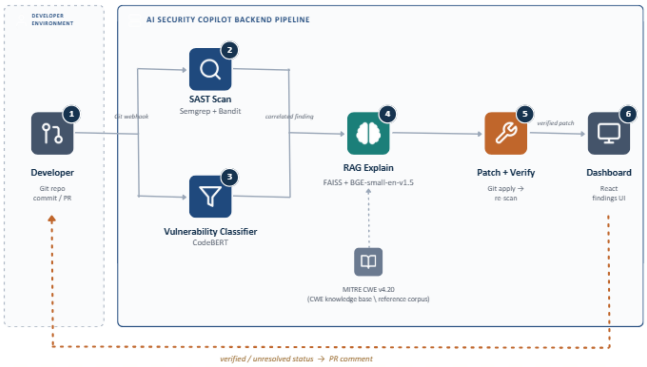


Fig. 1: System architecture of the proposed AI-assisted security copilot.

This system starts with the web-based user interface where developers can upload the source code for analysis. Source code analysis takes place via two components which complement each other, that is, rule-based static code analysis layer via Semgrep and Bandit tools, and fine-tuned CodeBERT model for detection of vulnerabilities. These results are correlated in order to receive a comprehensive vulnerability assessment. The security information is further retrieved by the RAG module with the help of CWE and OWASP security references. The retrieved information together with detected

vulnerabilities and findings of the analysis is used by the generative AI component in order to generate contextual explanation, recommendations for security and secure code patches.

The system proceeds to the next stage which is verification where the patch generated by the system is re-analyzed via the vulnerability detection pipeline to see whether the identified security problem is resolved. The final results that include vulnerability assessment, SAST findings, security recommendations, the generated patch and the verification result are provided via React-based dashboard.

B. Dataset Structure and Preprocessing

The *PyCode-Vul* dataset was used as the primary dataset for training the vulnerability detection model. It contains Python source-code samples with corresponding vulnerability labels and supporting metadata, including vulnerability identifiers, CWE/CVE information, repository details, file paths, and commit information. The model received only the source-code content as input and learned to predict the associated vulnerability class.

The dataset was inspected for missing values, duplicates, and potential data leakage. Source-code and label fields were standardized, and samples with missing or empty source code were removed. Fields with source codes and labels were pre-processed and samples with the missing or empty source code were excluded from the dataset. The source codes were also preprocessed for detecting and excluding duplicated functions, with comparisons made across different parts of the dataset to exclude overlapping samples. The cleaned dataset was divided into training, validation, and test sets. The training data was further split using an 85:15 stratified split with a fixed random seed of 42, while the test set remained isolated throughout model development. After cleaning and partitioning, the resulting distribution of functions, vulnerability classes, and generated code chunks across the training, validation, and test sets is presented in Table I.

TABLE I: PyCode-Vul Preprocessing and Chunking

Split	Functions	Non-Vulnerable	Vulnerable	Chunks
Training	12,025	6,742	5,283	36,301
Validation	2,123	1,190	933	6,220
Test	3,528	1,957	1,571	10,552
Total	**17,676**	**9,889**	**7,787**	**53,073**

C. Vulnerability Classifier

The vulnerability classification module is developed based on the CodeBERT model, which is a pre-trained transformer model that learns from both natural language and programming languages. In our solution, the CodeBERT model is fine-tuned as a binary sequence classification task that separates the source-code functions into vulnerable or non-vulnerable categories.

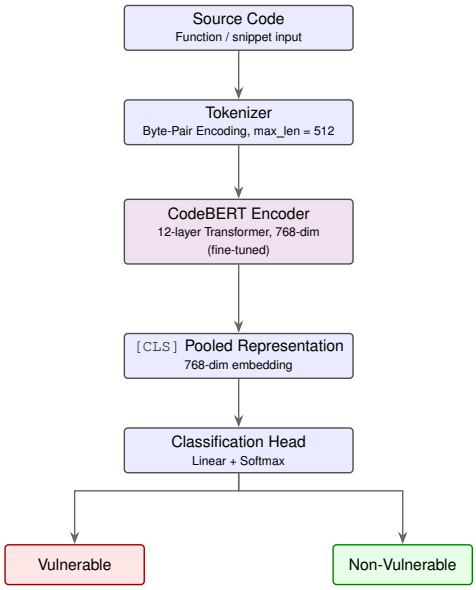


Fig. 2: Architecture of the fine-tuned CodeBERT-based vulnerability classification model.

1) CodeBERT Architecture: Vulnerability classification makes use of the pre-trained `microsoft/codebert-base` model as the encoder. The CodeBERT model is further fine-tuned using the vulnerability dataset with the addition of a binary classification layer, which is responsible for classifying whether the code is vulnerable or not. The overall architecture of the CodeBERT-based vulnerability classifier is illustrated in Fig. 2. Source code is tokenized, encoded by a fine-tuned CodeBERT transformer, pooled using the `[CLS]` token representation, and passed through a classification head to produce a binary vulnerable/non-vulnerable prediction.

2) Long Function Chunking: CodeBERT supports a maximum input length of 512 tokens. Since some source-code functions exceed this limit, directly truncating them could remove vulnerability-relevant information. Therefore, a sliding-window approach was used to divide long functions into smaller chunks. Each function was divided into chunks of up to 512 tokens with an overlap of 128 tokens between consecutive chunks. A maximum of 16 chunks was retained for each function. For functions exceeding this limit, chunks were selected approximately uniformly across the complete token span to preserve information from different parts of the function. Each chunk retained its corresponding function identifier and vulnerability label. During inference, chunk-level predictions were aggregated at the function level using the mean predicted probability of the vulnerable class.

D. Retrieval-Augmented Generation

The proposed AI-Security-Copilot incorporates a RAG pipeline to provide security-specific context for vulnerabilities identified during code analysis. The knowledge base is constructed from MITRE CWE version 4.20, contain-

ing structured information such as vulnerability descriptions, impacts, detection methods, mitigations, and examples. The retrieved knowledge is provided as contextual information to the downstream language model for vulnerability explanation and remediation.

1) Knowledge Base: The CWE version 4.20 XML dataset is parsed into structured records containing CWE identifiers and related security information, including descriptions, consequences, detection methods, mitigations, and examples. These records are divided into retrieval-oriented chunks, while deprecated entries are excluded. The resulting chunks form the security knowledge corpus.

2) Embedding and Retrieval: The knowledge chunks are converted into vector representations using the `BAAI/bge-small-en-v1.5` embedding model and stored in a FAISS `IndexFlatIP` vector index. When a query is received, it is embedded using the same model and used to retrieve semantically relevant knowledge. The system also identifies an explicit CWE identifier and the user's intent, such as explanation, detection, or remediation. These signals are used alongside semantic similarity to re-rank the retrieved candidates. The highest-ranked and diverse results are selected to construct the final context supplied to the language model.

3) Context Construction: The selected retrieval results have been put together to form a structured context that includes CWE information such as description, impact, detection, remediation, and examples. Duplicated information has been eliminated from the context, which has also been constrained to provide an appropriate input size for the generative model. This provides a context for combining information from the vulnerability detection process and CWE knowledge for the generative model.

E. Model Training

Following cleaning and leakage-analysis, PyCode-Vul dataset served as a data source to fine-tune the pre-trained `microsoft/codebert-base` model for binary vulnerability classification. Training was performed using the generated code chunks, while function identifiers were retained to enable function-level prediction during evaluation. The test set was excluded from training and model selection and was used only for final evaluation.

1) Training Configuration: The `microsoft/codebert-base` model was initialized with a binary classification head to categorize each function as vulnerable and non-vulnerable source-code functions. The training configuration and hyperparameters used for fine-tuning are presented in Table II. The model's performance was observed after each training epoch, with validation F1-score used as the model-selection criterion. The checkpoint achieving the highest validation F1-score was retained as the final model, while early stopping with a patience of two epochs was used to prevent unnecessary training. Dynamic padding was applied during training, and FP16 was enabled when CUDA acceleration was available. A fixed random seed was used to support reproducibility.

TABLE II: CodeBERT Training Configuration

Parameter	Value
Pre-trained model	`microsoft/codebert-base`
Task	Binary classification
Number of labels	2
Maximum sequence length	512 tokens
Epochs	10
Learning rate	2×10^{-5}
Weight decay	0.01
Warmup ratio	0.1
Training batch size	8
Evaluation batch size	16
Gradient accumulation	2 steps
Effective training batch size	16
Evaluation frequency	Every epoch
Best-model criterion	Validation F1
Early stopping patience	2 epochs
FP16	Enabled with CUDA
Random seed	42

2) Training Procedure: During training, the generated code chunks were provided to the CodeBERT encoder in batches. For each chunk, the model generated logits for the vulnerable and non-vulnerable classes. The class with the highest logit was selected as the prediction, while the predicted probability of the vulnerable class was retained for evaluation and function-level aggregation.

IV. Results and Discussion

The proposed vulnerability classification approach was evaluated on the held-out test set using accuracy, precision, recall, F1-score, ROC-AUC, and average precision. CodeBERT was compared with Logistic Regression (LR) and Random Forest (RF) baselines using the same test set.

A. Vulnerability Classification Performance

The fine-tuned CodeBERT model achieved 96.88% accuracy, 95.76% precision, 97.38% recall, and 96.56% F1-score as shown in Table III. The ROC-AUC of 0.9933 and average precision of 0.9931 indicate strong discrimination between vulnerable and non-vulnerable functions. The high recall is particularly relevant to vulnerability detection, where false negatives represent vulnerabilities that remain undetected.

TABLE III: CodeBERT Classification Performance

Metric	Score
Accuracy	96.88%
Precision	95.76%
Recall	97.38%
F1-Score	96.56%
ROC-AUC	99.33%
Average Precision	99.31%

B. Comparative Analysis with Baseline Models

To evaluate the benefit of contextual code representations, CodeBERT was compared with Logistic Regression and Random Forest using the same 3,563-sample held-out test set. The conventional models used TF-IDF features, while CodeBERT

was fine-tuned directly on source-code sequences, with the performance of all three models presented in Table IV.

TABLE IV: Comparison of Vulnerability Detection Models

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	94.19%	94.23%	92.75%	93.48%	98.68%
Random Forest	**95.65%**	**96.35%**	93.88%	95.10%	99.23%
CodeBERT	**96.88%**	95.76%	**97.38%**	**96.56%**	**99.33%**

CodeBERT achieved the strongest overall performance, with the highest accuracy (96.88%), F1-score (96.56%), and ROC-AUC (99.33%). Its F1-score exceeded Logistic Regression and Random Forest by 3.08 and 1.46 percentage points, respectively, indicating the advantage of contextual code representations over TF-IDF-based features.

CodeBERT also achieved the highest recall (97.38%) and only 42 false negatives. Although Random Forest achieved slightly higher precision, CodeBERT provided a stronger balance between precision and recall. Since false negatives represent undetected vulnerable functions, CodeBERT's higher recall supports its selection as the vulnerability classification component of the proposed system.

C. Error and Threshold Analysis

The confusion matrices in Figure 3 shows that CodeBERT produced 69 false positives and 42 false negatives, compared with 57 and 98 for Random Forest and 91 and 116 for Logistic Regression, respectively. Random Forest produced fewer false positives because its feature-based classification was more conservative, but this also caused it to miss significantly more actual vulnerabilities. On the other hand, the number of false negatives was significantly reduced by CodeBERT, which is especially crucial for security-based vulnerability detection. The error distribution reveals one significant difference between the models, as CodeBERT decreased the number of false negatives by 56.9% compared to Logistic Regression and by 57.1% compared to Random Forest.

Table V summarizes the prediction errors of the three models.

TABLE V: False-Positive and False-Negative Comparison

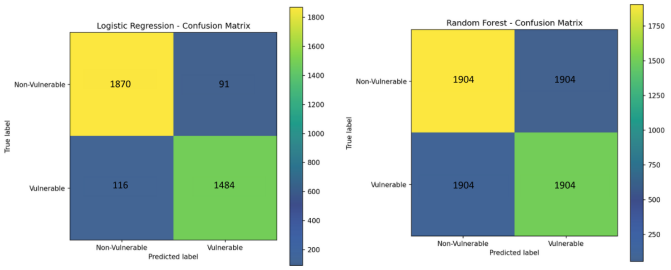
Model	False Positives	False Negatives
Logistic Regression	91	116
Random Forest	57	98
CodeBERT	69	42

D. ROC-AUC and Precision-Recall Analysis

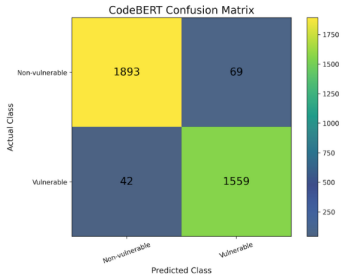
The ROC and precision-recall curves further demonstrate the strong classification performance of CodeBERT, with an ROC-AUC of 0.9933 and average precision of 0.9931. The curves indicate strong separation between the vulnerable and non-vulnerable classes across classification thresholds.

E. Ablation Study

To determine the individual impact of the components incorporated into the proposed framework, an ablation experiment was carried out. There were three methodologies used for



(a) Logistic Regression (b) Random Forest



(c) CodeBERT

Fig. 3: Confusion matrices of the evaluated vulnerability classification models: (a) Logistic Regression, (b) Random Forest, and (c) CodeBERT.

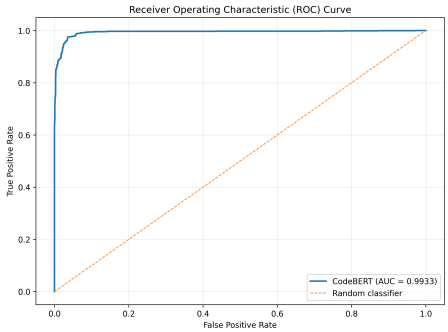


Fig. 4: ROC curve of the CodeBERT vulnerability classifier.

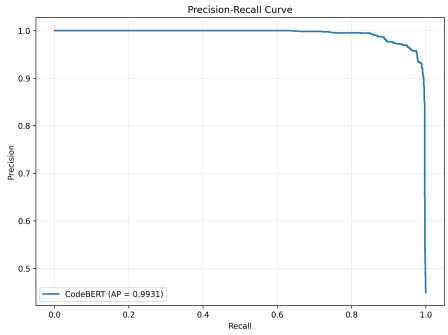


Fig. 5: Precision-recall curve of the CodeBERT vulnerability classifier.

this assessment: the CodeBERT methodology alone, the static analysis setup using Semgrep and Bandit, and the combination of both CodeBERT and SAST methods. All methodologies were evaluated under equal conditions using the same test data, and metrics like Accuracy, Precision, Recall, F1-Score, and ROC-AUC were used.

TABLE VI: Ablation Study Results

Configuration	Accuracy	Precision	Recall	F1	ROC-AUC
CodeBERT Only	96.88	95.76	97.38	96.56	99.33
SAST Only	59.65	68.68	12.00	21.09	55.26
CodeBERT + SAST	95.65	93.55	97.00	95.24	97.58

The results in Table VI demonstrated that CodeBERT-only achieved the strongest overall performance, with an accuracy of 96.88%, recall of 97.38%, F1-score of 96.56%, and ROC-AUC of 99.33%. In contrast, SAST-only achieved lower recall and F1-score, indicating that rule-based analysis missed a considerable number of vulnerabilities. The hybrid CodeBERT + SAST configuration achieved a high recall of 97.00% and an F1-score of 95.24%, demonstrating that the integration of static analysis provides complementary detection capabilities. Although the hybrid configuration did not outperform CodeBERT-only in the evaluated classification metrics, its integration within the proposed security copilot provides an additional rule-based analysis layer that can support the detection and validation of known insecure coding patterns.

V. Limitations and Future Scope

A. Limitations

The proposed framework is dependent on the coverage and accuracy of the underlying SAST tools and may produce false positives or miss vulnerabilities that require deeper program-flow or runtime analysis. The CodeBERT classifier is primarily trained on Python vulnerability data, which may limit its generalization to other programming languages and unseen vulnerability patterns. The RAG component is also dependent on the quality and coverage of the CWE knowledge base, and semantically related weaknesses may occasionally introduce retrieval noise. The current evaluation also covers a limited set of vulnerability categories and test cases, limiting the generalizability of the reported results.

B. Future Scope

Future work can extend the framework to support multiple programming languages by incorporating language-specific SAST tools and vulnerability classification models. Moreover, the RAG knowledge base can be enriched with continuously updated CWE, CVE/NVD, OWASP, and MITRE ATT&CK data in addition to enhanced retrieval and reranking techniques. The future work could also include the integration of CI/CD pipelines to provide continuous security analysis during software development. Additionally, incorporating developer feedback and adaptive learning can help reduce false positives and improve the system's detection and remediation capabilities.

VI. Conclusion

This work presented an AI-assisted security copilot for detecting and mitigating code vulnerabilities through combining static analysis, machine learning, and RAG into a unified workflow. Static analysis is used to identify potential security issues and along with it, the machine-learning component provides an additional assessment of vulnerable source-code functions. Among the evaluated models, CodeBERT achieved the best overall performance with 96.88% accuracy, 97.38% recall, 96.56% F1-score, and 99.33% ROC-AUC, demonstrating its suitability for contextual source-code vulnerability classification. The proposed copilot also leverages RAG to provide context-based explanations and remediation guidance for detected vulnerabilities, reducing the need for manual analysis of security findings. By combining automated detection, contextual classification, and actionable security guidance, the system moves beyond conventional vulnerability scanners toward an integrated AI-assisted security analysis workflow. Future work can focus on supporting additional vulnerability types and programming languages, improving false-positive reduction, expanding the security knowledge base, and evaluating the system with larger real-world codebases and security practitioners.

References

[1] A. Kharkar et al., "Learning to Reduce False Positives in Analytic Bug Detectors," in *Proc. ICSE*, 2022, https://arxiv.org/abs/2203.09907

[2] Y. Xiong and T. Zhang, "Sifting the Noise: A Comparative Study of LLM Agents in Vulnerability False Positive Filtering," arXiv:2601.22952, 2026, https://arxiv.org/abs/2601.22952

[3] X. Du et al., "Reducing False Positives in Static Bug Detection with LLMs: An Empirical Study in Industry," arXiv:2601.18844, 2026, https://arxiv.org/abs/2601.18844

[4] M. Iranmanesh et al., "ZeroFalse: Improving Precision in Static Analysis with LLMs," arXiv:2510.02534, 2025, https://arxiv.org/abs/2510.02534

[5] Z. Jiang et al., "DFEPT: Data Flow Embedding for Enhancing Pre-Trained Model Based Vulnerability Detection," in *Proc. Internetware*, 2024, pp. 95–104, https://doi.org/10.1145/3671016.3671388

[6] D. Guo et al., "GraphCodeBERT: Pre-Training Code Representations with Data Flow," in *Proc. ICLR*, 2021, https://arxiv.org/abs/2009.08366

[7] Z. Xiong and W. Dong, "VulD-CodeBERT: CodeBERT-Based Vulnerability Detection Model for C/C++ Code," 2024, https://www.researchgate.net/publication/383898284

[8] Y. Jiang et al., "StagedVulBERT: Multi-Granular Vulnerability Detection with a Novel Pre-Trained Code Model," *IEEE Trans. Software Eng.*, vol. 50, no. 12, 2024, https://arxiv.org/abs/2410.05766

[9] R. Safdar et al., "Data and Context Matter: Towards Generalizing AI-Based Software Vulnerability Detection," arXiv:2508.16625, 2025, https://arxiv.org/abs/2508.16625

[10] Y. Chen et al., "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection," in *Proc. RAID*, 2023, pp. 654–668, https://arxiv.org/abs/2304.00409

[11] Y. Ding et al., "Vulnerability Detection with Code Language Models: How Far Are We?," in *Proc. ICSE*, 2025, pp. 469–481, https://arxiv.org/abs/2403.18624

[12] L. Wartschinski et al., "VUDENC: Vulnerability Detection with Deep Learning on a Natural Codebase for Python," *Inf. Softw. Technol.*, vol. 144, p. 106809, 2022, https://arxiv.org/abs/2201.08441

[13] S-AiRLab, "PyCode-Vul: A Benchmark Dataset for Python," Kaggle, 2026, https://www.kaggle.com/datasets/oulabspring/pycode-vul-a-benchmark-dataset-for-python

[14] X. Du et al., "Vul-RAG: Enhancing LLM-Based Vulnerability Detection via Knowledge-Level RAG," arXiv:2406.11147, 2024, https://arxiv.org/abs/2406.11147

[15] M. Fu et al., "VulRepair: A T5-Based Automated Software Vulnerability Repair," in *Proc. ESEC/FSE*, 2022, https://doi.org/10.1145/3540250.3549098

[16] A. Al-Maamari, "Why LLMs Fail: A Failure Analysis and Partial Success Measurement for Automated Security Patch Generation," arXiv:2603.10072, 2026, https://arxiv.org/abs/2603.10072

[17] C. Cheng, "Detect-Repair-Verify for LLM-Generated Code: A Multi-Language, Multi-Granularity Empirical Study," arXiv:2603.23633, 2026, https://arxiv.org/abs/2603.23633